Majority Element

I) Using HashMap

nums = [5 , 5 , 1 , 1 , 1 , 5 , 5] , n = 7

→ We can use an hashmap to keep track of frequency of each element as we iterate.
→ Along any point, if the frequency of a certain element exceeds n/2, we can simply exit the loop, as this is our majority element.

T.C. - O(N)
S.C. - O(N)

II) Using Moore's Voting Algorithm

nums = [5 , 5 , 1 , 1 , 1 , 5 , 5] , n = 7

→ The idea is to assume that the first element of the array is a candidate for the majority element
→ We will also keep a count variable which keeps track of the frequency of the current candidate.
→ If along the iteration, an element encountered is not the current candidate we decrement the count
→ At any moment, when the count of a candidate becomes zero, we update the candidate to the current element in iteration, and initialize count as 1

→ When the iteration is complete, the candidate is the only possible value which can be the majority element.

→ In the second iteration, while verifying the candidate, if it's not greater than n/2, it means that no majority element exists for given array

```java
class Solution {
    public int majorityElement(int[] nums) {
        // Using voting algorithm
        int candidate = nums[0];
        int count = 1;

        for (int i = 1; i < nums.length; i++) {
            if (nums[i] == candidate) {
                count++;
            } else {
                count--;
                if (count == 0) {
                    candidate = nums[i];
                    count = 1;
                }
            }
        }

        return candidate;
    }
}
```